

residual dropout Srivastava et al. (2014), weight decay Loshchilov & Hutter (2017) and gradient noise Neelakantan et al. (2015). The results are shown in Figure 2 (left).

We find that adding weight decay has a very large effect on data efficiency, more than halving the amount of samples needed compared to most other interventions. We found that weight decay towards the initialization of the network is also effective, but not quite as effective as weight decay towards the origin. This makes us believe that the prior, that approximately zero weights are suitable for small algorithmic tasks, explains part, but not all of the superior performance of weight decay. Adding some noise to the optimization process (e.g. gradient noise from using minibatches, Gaussian noise applied to weights before or after computing the gradients) is beneficial for generalization, consistent with the idea that such noise might induce the optimization to find flatter minima that generalize better. We found that learning rate had to be tuned in a relatively narrow window for the generalization to happen (within 1 order of magnitude).

3.4 QUALITATIVE VISUALIZATION OF EMBEDDINGS

In order to gain some insight into networks that generalize, we visualized the matrix of the output layer for the case of modular addition and S_5 . In Figure 3 we show t-SNE plots of the row vectors. For some networks we find clear reflections of the structure of the underlying mathematical objects in the plots. For example the circular topology of modular addition is shown with a ‘number line’ formed by adding 8 to each element. The structure is more apparent in networks that were optimized with weight decay.

4 DISCUSSION

We have seen that in the datasets we studied, small algorithmic binary operation tables, effects such as double descent or late generalization, and improvements to generalization from interventions like weight decay can be striking. This suggests that these datasets could be a good place to investigate aspects of generalization. For example, we plan to test whether various proposed measures of minima flatness correlate with generalization in our setting.

We have also seen that visualizing the embedding spaces of these neural networks can show natural kinds of structure, for example in problems of modular arithmetic the topology of the embeddings tends to be circles or cylinders. We also see that the network tends to idiosyncratically organize the embeddings by various residues. Whilst the properties of these mathematical objects are familiar to us, we speculate that such visualizations could one day be a useful way to gain intuitions about novel mathematical objects.

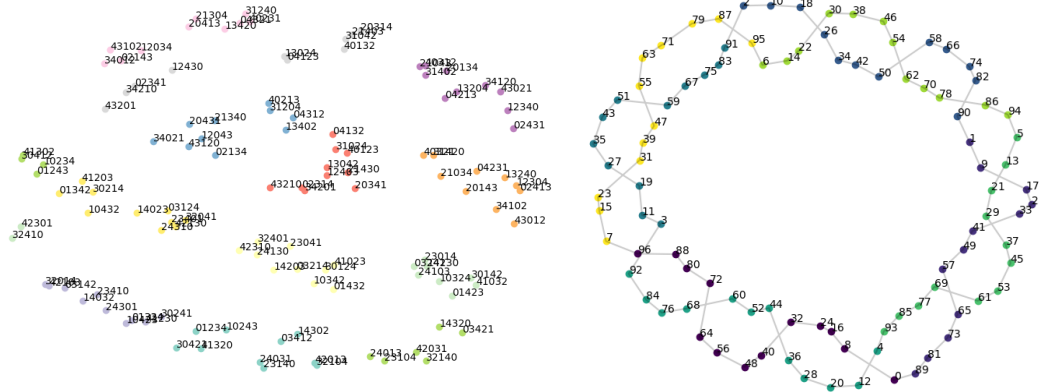


Figure 3: **Left.** t-SNE projection of the output layer weights from a network trained on S_5 . We see clusters of permutations, and each cluster is a coset of the subgroup $\langle (0, 3)(1, 4), (1, 2)(3, 4) \rangle$ or one of its conjugates. **Right.** t-SNE projection of the output layer weights from a network trained on modular addition. The lines show the result of adding 8 to each element. The colors show the residue of each element modulo 8.

In addition, we document an interesting phenomenon, where the number of optimization steps needed to reach a given level of performance increases quickly as we reduce the size of the training dataset. Since this represents a way trade compute for performance on smaller amounts of data, it would be useful to investigate in future work whether the effect is also present for other datasets.

REFERENCES

- Mikhail Belkin, Daniel Hsu, Siyuan Ma, and Soumik Mandal. Reconciling modern machine learning practice and the bias-variance trade-off. *arXiv preprint arXiv:1812.11118*, 2018.
- Stéphane d’Ascoli, Levent Sagun, and Giulio Biroli. Triple descent and the two kinds of overfitting: Where & why do they appear? *arXiv preprint arXiv:2006.03509*, 2020.
- Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Łukasz Kaiser. Universal transformers. *arXiv preprint arXiv:1807.03819*, 2018.
- Alex Graves. Adaptive computation time for recurrent neural networks. *arXiv preprint arXiv:1603.08983*, 2016.
- Alex Graves, Greg Wayne, and Ivo Danihelka. Neural turing machines. *arXiv preprint arXiv:1410.5401*, 2014.
- Edward Grefenstette, Karl Moritz Hermann, Mustafa Suleyman, and Phil Blunsom. Learning to transduce with unbounded memory. *Advances in neural information processing systems*, 28: 1828–1836, 2015.
- Sepp Hochreiter and Jürgen Schmidhuber. Flat minima. *Neural computation*, 9(1):1–42, 1997.
- Yiding Jiang, Behnam Neyshabur, Hossein Mobahi, Dilip Krishnan, and Samy Bengio. Fantastic generalization measures and where to find them. *arXiv preprint arXiv:1912.02178*, 2019.
- Łukasz Kaiser and Ilya Sutskever. Neural gpus learn algorithms. *arXiv preprint arXiv:1511.08228*, 2015.
- Nitish Shirish Keskar, Dheevatsa Mudigere, Jorge Nocedal, Mikhail Smelyanskiy, and Ping Tak Peter Tang. On large-batch training for deep learning: Generalization gap and sharp minima. *CoRR*, abs/1609.04836, 2016. URL <http://arxiv.org/abs/1609.04836>.
- Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017.
- Preetum Nakkiran, Gal Kaplun, Yamini Bansal, Tristan Yang, Boaz Barak, and Ilya Sutskever. Deep double descent: Where bigger models and more data hurt. *arXiv preprint arXiv:1912.02292*, 2019.
- Arvind Neelakantan, Luke Vilnis, Quoc V Le, Ilya Sutskever, Łukasz Kaiser, Karol Kurach, and James Martens. Adding gradient noise improves learning for very deep networks. *arXiv preprint arXiv:1511.06807*, 2015.
- Scott Reed and Nando De Freitas. Neural programmer-interpreters. *arXiv preprint arXiv:1511.06279*, 2015.
- David Saxton, Edward Grefenstette, Felix Hill, and Pushmeet Kohli. Analysing mathematical reasoning abilities of neural models. *arXiv preprint arXiv:1904.01557*, 2019.
- Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. *The journal of machine learning research*, 15(1):1929–1958, 2014.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *arXiv preprint arXiv:1706.03762*, 2017.
- Jason Weston, Sumit Chopra, and Antoine Bordes. Memory networks. *arXiv preprint arXiv:1410.3916*, 2014.

Jason Weston, Antoine Bordes, Sumit Chopra, Alexander M Rush, Bart van Merriënboer, Armand Joulin, and Tomas Mikolov. Towards ai-complete question answering: A set of prerequisite toy tasks. *arXiv preprint arXiv:1502.05698*, 2015.

Wojciech Zaremba and Ilya Sutskever. Reinforcement learning neural turing machines-revised. *arXiv preprint arXiv:1505.00521*, 2015.

Chiyuan Zhang, Samy Bengio, Moritz Hardt, Benjamin Recht, and Oriol Vinyals. Understanding deep learning requires rethinking generalization. *arXiv preprint arXiv:1611.03530*, 2016.

A APPENDIX

A.1 ADDITIONAL EXPERIMENTAL DETAILS

A.1.1 BINARY OPERATIONS

The following are the binary operations that we have tried (for a prime number $p = 97$):

$$\begin{aligned}
x \circ y &= x + y \pmod{p} \text{ for } 0 \leq x, y < p \\
x \circ y &= x - y \pmod{p} \text{ for } 0 \leq x, y < p \\
x \circ y &= x/y \pmod{p} \text{ for } 0 \leq x < p, 0 < y < p \\
x \circ y &= \lfloor x/y \rfloor \pmod{p} \text{ if } y \text{ is odd, otherwise } x - y \pmod{p} \text{ for } 0 \leq x, y < p \\
x \circ y &= x^2 + y^2 \pmod{p} \text{ for } 0 \leq x, y < p \\
x \circ y &= x^2 + xy + y^2 \pmod{p} \text{ for } 0 \leq x, y < p \\
x \circ y &= x^2 + xy + y^2 + x \pmod{p} \text{ for } 0 \leq x, y < p \\
x \circ y &= x^3 + xy \pmod{p} \text{ for } 0 \leq x, y < p \\
x \circ y &= x^3 + xy^2 + y \pmod{p} \text{ for } 0 \leq x, y < p \\
x \circ y &= x \cdot y \text{ for } x, y \in S_5 \\
x \circ y &= x \cdot y \cdot x^{-1} \text{ for } x, y \in S_5 \\
x \circ y &= x \cdot y \cdot x \text{ for } x, y \in S_5
\end{aligned}$$

For each binary operation we constructed a dataset of equations of the form $\langle x \rangle \langle op \rangle \langle y \rangle \langle = \rangle \langle x \circ y \rangle$, where $\langle a \rangle$ stands for the token corresponding to element a .

For each training run, we chose a fraction of all available equations at random and declared them to be the training set, with the rest of equations being the validation set.

A.1.2 MODEL AND OPTIMIZATION

We trained a standard decoder-only transformer Vaswani et al. (2017) with causal attention masking, and calculated loss and accuracy only on the answer part of the equation. For all experiments we used a transformer with 2 layers, width 128, and 4 attention heads, with a total of about $4 \cdot 10^5$ non-embedding parameters.

We have tuned optimization hyperparameters by running experiments on modular addition and product in S_5 . For final configuration of hyperparameters we have chosen a balance of performance we saw on S_5 and simplicity (for example we chose not to anneal the learning rate for the experiments in the paper even though it performed better in some situations). For most experiments we used AdamW optimizer with learning rate 10^{-3} , weight decay 1, $\beta_1 = 0.9$, $\beta_2 = 0.98$, linear learning rate warmup over the first 10 updates, minibatch size 512 or half of training dataset size (whichever was smaller) and optimization budget of 10^5 gradient updates.

In section 3.3 we have also tried the following variants (listed in the reading order for Figure 2 left):

- Adam optimizer with full batch (i.e. exact gradient of the loss on the whole training dataset)
- Adam optimizer